National University of Computer & Emerging Sciences

Computer Science Department

Course Code: CL-2006	**Course:** Operating System Lab
Instructor: Yasir Arfat	**Semester:** Spring 2025

Lab Manual – 06

Inter-Process Communication (IPC)
File Descriptors, Pipes & Named Pipes (FIFO)

Table of Contents

Objectives

This lab introduces Inter-Process Communication (IPC) — the mechanisms that allow processes to exchange data and coordinate their execution. Building on the process management concepts from Lab 05 (fork, exec, wait), you will now learn how processes actually talk to each other. Upon completion, students will be able to:

- Understand what IPC is and why it is essential in operating systems
- Explain the role of file descriptors in all Linux I/O and process communication
- Understand how the kernel manages file descriptors internally (per-process FD table, open file table)
- Use file descriptor system calls: open(), close(), read(), write(), dup(), dup2()
- Redirect standard I/O (stdin, stdout, stderr) using dup2() in C programs
- Create unnamed pipes using the pipe() system call for parent–child communication
- Implement bidirectional communication between parent and child processes
- Use named pipes (FIFO) for communication between unrelated processes
- Compare unnamed pipes and named pipes and choose the appropriate one

Concepts

What is Inter-Process Communication (IPC)?

In Lab 05, you learned how to create new processes using fork(). But creating processes is only half the story — in real systems, processes need to work together. A web server might fork a child to handle each request, a compiler might split work across multiple processes, or a media player might have separate processes for decoding audio and video.

Inter-Process Communication (IPC) is the set of mechanisms provided by the operating system that allow processes to exchange data and synchronize their actions. Without IPC, each process would be completely isolated in its own address space with no way to collaborate.

> **OS CONCEPT**
> Every process has its own virtual address space, so one process cannot directly read or write another process's memory. The OS kernel acts as an intermediary — it provides controlled communication channels (pipes, queues, shared memory regions) that processes can use to exchange data safely. This is a fundamental design decision in modern operating systems: isolation by default, communication by explicit request.

Linux provides several IPC mechanisms, each suited for different scenarios:

IPC Method	Direction	Related Processes?	Best Used For
Unnamed Pipe	Unidirectional	Yes (parent–child)	Simple data streaming between related processes
Named Pipe (FIFO)	Unidirectional	No (any processes)	Cross-process messaging via filesystem
Message Queue	Bidirectional	No (any processes)	Structured messages with priority (Lab 07)
Shared Memory	Bidirectional	No (any processes)	Large data sharing, fastest IPC (Lab 07)
Signals	Unidirectional	No	Notifications / interrupts
Semaphores	N/A (sync only)	No	Synchronization / mutual exclusion
Sockets	Bidirectional	No (even network)	Network communication

In this lab, we will focus on file descriptors, unnamed pipes, and named pipes (FIFO). These are the building blocks — every other IPC mechanism also relies on file descriptors underneath. Message queues and shared memory will be covered in Lab 07.

File Descriptors – The Foundation of IPC

Before diving into pipes, you must understand file descriptors — they are the key to how Linux handles all I/O, including inter-process communication. Every pipe, every FIFO, every socket you will ever use in this course is ultimately just a pair of file descriptors.

What is a File Descriptor?

A file descriptor (FD) is a small non-negative integer that the operating system assigns to every open file, pipe, socket, or device. Think of it as a ticket number at a bank — when you open a file, the kernel gives you a number, and you use that number for all future operations (reading, writing, closing) on that resource.

OS CONCEPT

In Linux, the philosophy is "everything is a file." Regular files, directories, pipes, sockets, terminals, and even hardware devices are all accessed through the same interface: file descriptors. This unified design is what makes IPC mechanisms like pipes work seamlessly — a pipe is just a pair of file descriptors, one for reading and one for writing. The same read() and write() system calls work on files, pipes, sockets, and devices.

The Three Standard File Descriptors

Every process in Linux automatically starts with three file descriptors already open. These are inherited from the parent process (usually the shell):

FD Number	Name	C Constant	Purpose	Default Connection
0	Standard Input (stdin)	STDIN_FILENO	Reading input data	Keyboard / terminal
1	Standard Output (stdout)	STDOUT_FILENO	Writing normal output	Terminal screen
2	Standard Error (stderr)	STDERR_FILENO	Writing error messages	Terminal screen

When you call printf() in C, it internally calls write() on file descriptor 1 (stdout). When you call scanf(), it calls read() on file descriptor 0 (stdin). When you call perror() or fprintf(stderr, ...), it writes to file descriptor 2 (stderr).

TIP

Remember from Lab 02: shell redirection operators (>, <, 2>) work by manipulating file descriptors behind the scenes. The command ./program > output.txt tells the shell to redirect FD 1 (stdout) to a file before starting the program. The command ./program 2> errors.log redirects FD 2 (stderr). Now you understand the kernel mechanism behind what the shell does!

How File Descriptors Work Internally

Inside the kernel, each process has a file descriptor table — an array where each index is a file descriptor number and each entry points to an open file description in the kernel's global open file table. Here is the step-by-step flow when you open a file:

1. Your program calls open("/home/student/data.txt", O_RDONLY).
2. The kernel searches for the lowest available FD number in your process's FD table (usually 3, since 0, 1, 2 are taken).
3. The kernel creates an entry in its global open file table with: the current read/write position (offset), access mode (read-only, write-only, read-write), and a pointer to the file's inode (the actual data on disk).
4. The kernel stores a reference to this open file table entry in your process's FD table at index 3.
5. The integer 3 is returned to your program. You now use 3 for all operations on this file.

Why this matters for IPC: When fork() is called, the child process gets a COPY of the parent's entire file descriptor table. Both parent and child now share the same underlying open file descriptions in the kernel. This is exactly how pipes work — the parent creates a pipe (which gives two FDs), then calls fork(), and now both parent and child have FDs pointing to the same pipe. The parent writes to one end, the child reads from the other.

> **OS CONCEPT**
> The FD table is per-process, but the open file descriptions in the kernel are shared. After fork(), if the parent reads 100 bytes from a shared FD, the file offset advances for BOTH processes. This shared-offset behavior is important to understand for pipes and file sharing between parent and child.

File Descriptor System Calls

System Call	Header	Purpose
open(path, flags)	<fcntl.h>	Open a file or device, returns a new FD
close(fd)	<unistd.h>	Close a file descriptor, free the resource
read(fd, buf, count)	<unistd.h>	Read up to count bytes from FD into buffer
write(fd, buf, count)	<unistd.h>	Write count bytes from buffer to FD
dup(fd)	<unistd.h>	Duplicate FD (returns new FD pointing to same open file)
dup2(oldfd, newfd)	<unistd.h>	Duplicate oldfd onto newfd (closes newfd first if open)
pipe(int fd[2])	<unistd.h>	Create a pipe: fd[0] for reading, fd[1] for writing

Example 1: Seeing File Descriptors in Action

This program demonstrates how the kernel assigns file descriptors sequentially, and what happens when you close one and open another:

fd_demo.c

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    // These three are already open: 0 (stdin), 1 (stdout), 2 (stderr)
    printf("stdin  = %d\n", STDIN_FILENO);   // Always 0
    printf("stdout = %d\n", STDOUT_FILENO);   // Always 1
    printf("stderr = %d\n", STDERR_FILENO);   // Always 2

    // Open files — kernel picks the lowest available FD each time
    int fd1 = open("file_a.txt", O_CREAT | O_WRONLY, 0644);
    int fd2 = open("file_b.txt", O_CREAT | O_WRONLY, 0644);
    int fd3 = open("file_c.txt", O_CREAT | O_WRONLY, 0644);
    printf("file_a = %d, file_b = %d, file_c = %d\n", fd1, fd2, fd3);

    // Close file_b (FD 4) — this frees up FD 4
    close(fd2);

    // Open another file — kernel reuses the lowest free FD (4)
    int fd4 = open("file_d.txt", O_CREAT | O_WRONLY, 0644);
    printf("file_d = %d  (reused FD 4!)\n", fd4);

    close(fd1); close(fd3); close(fd4);
    return 0;
}
```

Output

```
stdin  = 0
stdout = 1
stderr = 2
file_a = 3, file_b = 4, file_c = 5
file_d = 4  (reused FD 4!)
```

NOTE

Notice how the kernel assigned FDs 3, 4, 5 sequentially. After closing FD 4 (file_b), the next open() call reused FD 4. The kernel always picks the lowest available number. This is not just trivia — dup2() exploits this behavior to redirect stdin/stdout.

Example 2: Redirecting stdout Using dup2()

The dup2() system call is how the shell implements output redirection (the > operator). It duplicates one FD onto another. Here we redirect stdout (FD 1) to a file, so all printf() output goes to the file instead of the screen:

dup2_demo.c

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    printf("This goes to the terminal.\n");

    // Open a file for writing
    int fd = open("output.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    // Redirect stdout (FD 1) to the file
    // dup2 closes the old FD 1 (terminal) and makes FD 1 point to the file
    dup2(fd, STDOUT_FILENO);
    close(fd);    // fd is no longer needed (FD 1 now points to the file)

    // From now on, ALL printf/write to stdout goes to output.txt
    printf("This goes to the file!\n");
    printf("So does this.\n");

    return 0;
}
```

```
$ gcc dup2_demo.c -o dup2_demo
$ ./dup2_demo
This goes to the terminal.
$ cat output.txt
This goes to the file!
So does this.
```

OS CONCEPT

This is exactly what the shell does when you type ./program > output.txt. The shell: (1) calls open() on output.txt, (2) calls dup2(fd, 1) to redirect stdout, (3) calls exec() to replace itself with your program. Your program never knows its stdout was redirected — it just writes to FD 1 as usual, and the data goes to the file.

Example 3: File Descriptors Survive fork()

This is the critical connection between file descriptors and IPC. When you fork(), the child inherits all open FDs:

fd_fork.c

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd = open("shared.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    int pid = fork();

    if (pid > 0) {
        // Parent writes first
        write(fd, "Parent wrote this\n", 18);
        wait(NULL);
        close(fd);
    }
    else if (pid == 0) {
        // Child also has FD pointing to the SAME file
        write(fd, "Child wrote this\n", 17);
        close(fd);
    }

    return 0;
}
```

```
$ gcc fd_fork.c -o fd_fork && ./fd_fork
$ cat shared.txt
Parent wrote this
Child wrote this
```

Both parent and child wrote to the same file using the same FD number! The kernel's shared open file description ensured the writes did not overwrite each other — the file offset advanced after each write.

WARNING

Always close file descriptors when you are done. Each process has a limited number of FDs (usually 1024 by default). Forgetting to close FDs causes resource leaks. In IPC with pipes, unclosed pipe ends can cause deadlocks — a read() will block forever waiting for data that will never come because the write end was never closed.

Unnamed Pipes

What is an Unnamed Pipe?

An unnamed pipe (or simply “pipe”) is the simplest IPC mechanism in Linux. It creates a one-way communication channel between two related processes (typically parent and child). Now that you understand file descriptors, a pipe is easy to understand: it is simply a kernel buffer with two file descriptors — one for reading (FD 0 of the array) and one for writing (FD 1 of the array).

Think of it like a water hose in a garden: water flows in one direction only — from the tap (write end) to the plants (read end). You cannot push water back through the same hose.

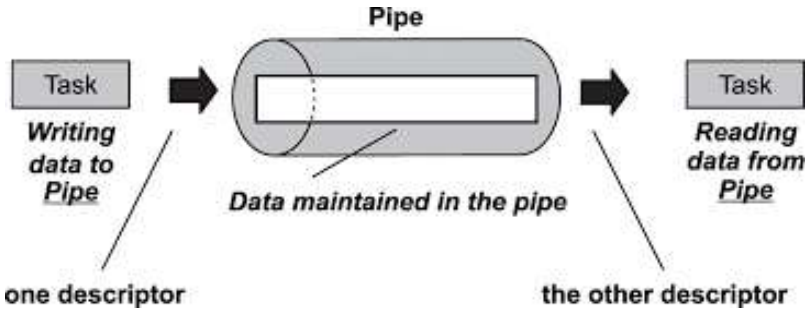

Figure 1: Unnamed pipe – data flows from the write end (FD 1) to the read end (FD 0) through a kernel buffer.

How pipe() Works with File Descriptors

The pipe() system call creates a pipe and returns two file descriptors in an integer array. This is where your file descriptor knowledge directly applies:

```
int pipefd[2];
pipe(pipefd);
// pipefd[0] = read  end  (data comes OUT of the pipe here)
// pipefd[1] = write end  (data goes INTO the pipe here)

// These are just regular file descriptors!
// You use read() and write() on them, just like files.
```

Array Index	File Descriptor	Direction	Same as...
pipefd[0]	Read end	Data comes OUT	Reading from a file with read(fd, ...)
pipefd[1]	Write end	Data goes IN	Writing to a file with write(fd, ...)

OS CONCEPT

A pipe is a kernel buffer (typically 64 KB on modern Linux systems). The kernel manages synchronization automatically: if the pipe is empty, read() blocks until data is available. If the pipe is full, write() blocks until space is freed. This automatic blocking behavior provides built-in synchronization between communicating processes — no extra code needed.

The pipe + fork pattern: (1) Create the pipe with pipe(). (2) Call fork(). Both parent and child now inherit the two FDs. (3) Each process closes the end it does not use. (4) One writes, the other reads. This is the standard pattern for pipe-based IPC.

Important rules for unnamed pipes:

- Unidirectional — data flows in one direction only (write end → read end).
- Related processes only — the pipe must be created BEFORE fork(), so both parent and child inherit the file descriptors.
- No persistence — once both ends are closed, the pipe and its data are gone.
- Always close unused ends — the parent should close the read end if writing, and the child should close the write end if reading. Failure to do this causes hangs.

Example 4: Basic Pipe – Parent Writes, Child Reads

In this example, the parent process sends a message to the child through a pipe. The child waits, reads the message, and prints it to the screen. Notice how we close unused pipe ends in each process:

unnamed_pipe.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void) {
    int pipefd[2];          // Array to hold the two pipe FDs
    char buffer[15];

    pipe(pipefd);           // Create pipe BEFORE fork!
    int pid = fork();       // Both parent and child now have pipefd[0] and pipefd[1]

    if (pid > 0) {
        // === PARENT PROCESS ===
        close(pipefd[0]);                       // Parent won't read, close read end
        printf("unnamed_pipe [INFO] Parent Process\n");
        write(pipefd[1], "Hello Mr.Linux", 15);  // Write message into pipe
        close(pipefd[1]);                       // Done writing, close write end
        wait(NULL);                             // Wait for child to finish
    }
```

```
    else if (pid == 0) {
        // === CHILD PROCESS ===
        close(pipefd[1]);                       // Child won't write, close write end
        sleep(2);                               // Simulate some delay
        printf("unnamed_pipe [INFO] Child Process\n");
        read(pipefd[0], buffer, sizeof(buffer));// Read from pipe (blocks if empty)
        printf("Received: %s\n", buffer);
        close(pipefd[0]);                       // Done reading, close read end
    }
    else {
        printf("unnamed_pipe [ERROR] fork failed\n");
    }
    return 0;
}
```

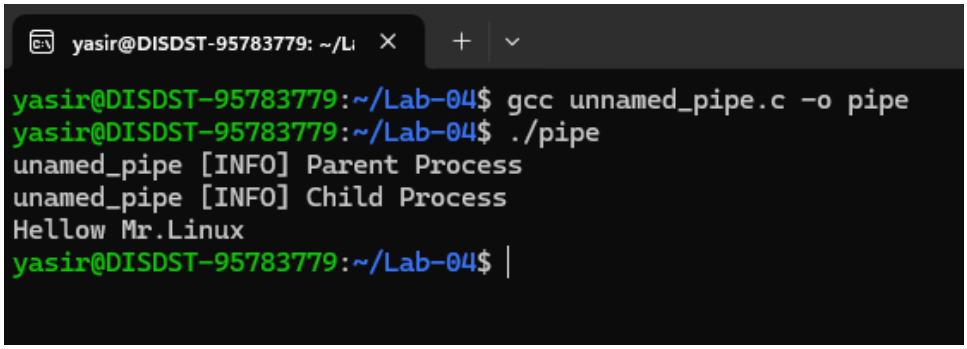

Figure 2: Output of unnamed_pipe.c – Parent writes, child reads after a 2-second delay.

NOTE

Notice how the child's read() automatically waits until the parent writes data. This is the built-in synchronization of pipes — no extra code needed. The kernel handles the blocking for you. Also notice we close unused ends: parent closes pipefd[0], child closes pipefd[1].

Example 5: Two-Way Communication with a Single Pipe

What if the child wants to send a reply back to the parent? You can reuse the same pipe if you carefully control the order of operations. The parent writes first and closes its write end, then the child reads, writes a reply, and closes its write end, and finally the parent reads the reply.

unnamed_pipe2.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
```

```
int main(void) {
    int pipefd[2];
    char buffer[15];

    pipe(pipefd);
    int pid = fork();

    if (pid > 0) {
        // === PARENT ===
        printf("unnamed_pipe [INFO] Parent Process\n");
        write(pipefd[1], "Hello Child", 12);
        close(pipefd[1]);            // Done writing, close write end
    }
    else if (pid == 0) {
        // === CHILD ===
        sleep(2);
        printf("unnamed_pipe [INFO] Child Process\n");
        read(pipefd[0], buffer, sizeof(buffer));
        close(pipefd[0]);            // Done reading parent's message
        printf("Received: %s\n", buffer);

        // Send reply back through the same pipe
        write(pipefd[1], "Hello Parent", 13);
        close(pipefd[1]);            // Done writing reply
        exit(0);                     // IMPORTANT: child must exit here!
    }
    else {
        printf("unnamed_pipe [ERROR] fork failed\n");
    }

    // Only PARENT reaches here (child exited above)
    wait(NULL);                      // Wait for child to finish
    read(pipefd[0], buffer, sizeof(buffer));
    close(pipefd[0]);
    printf("Reply from Child: %s\n", buffer);

    return 0;
}
```

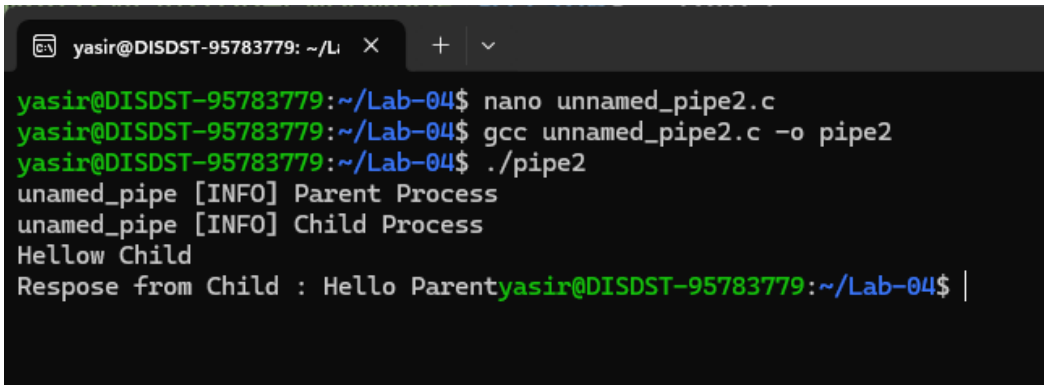

Figure 3: Output after fix – bidirectional communication using a single pipe with exit(0) in the child.

WARNING

The original code had a critical bug: without exit(0) at the end of the child branch, BOTH processes would execute the code after the if/else block, causing the reply message to print twice. Always use exit(0) at the end of a child process branch to prevent fall-through. This is one of the most common mistakes when using fork().

TIP

For robust bidirectional communication, it is better to use TWO separate pipes — one for parent→child and another for child→parent. This avoids the fragile timing dependencies of reusing a single pipe. We will see this pattern in the lab tasks.

Named Pipes (FIFO)

What is a Named Pipe?

The biggest limitation of unnamed pipes is that they only work between related processes (parent–child). This is because unnamed pipes have no name on the filesystem — the only way to share the FDs is through fork() inheritance.

A named pipe (also called a FIFO — First In, First Out) solves this problem. It is a special file that exists on the filesystem with a real path (like /tmp/myfifo). Any process that knows the path can open it for reading or writing, regardless of whether the processes are related.

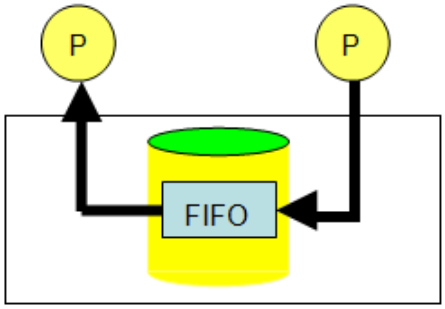

Figure 4: Named pipe (FIFO) – Two unrelated processes communicate through a shared FIFO file on the filesystem.

Real-world analogy: Think of a named pipe like a letterbox. A person (Process A) drops a letter into the box, and a completely different person (Process B) picks it up. They do not need to know each other — they just both know where the letterbox is. The letterbox has a physical address (the file path).

Unnamed Pipe vs Named Pipe

Feature	Unnamed Pipe	Named Pipe (FIFO)
Creation	pipe() system call	mkfifo() system call or mkfifo command
Filesystem presence	No — exists only in kernel memory	Yes — appears as a special file on disk
Process relationship	Must be related (parent–child)	Any processes can use it
Identification	By inherited file descriptors	By file path (e.g., /tmp/myfifo)
Persistence	Destroyed when both ends close	File persists until deleted with unlink()
Permissions	Inherited from parent	Set at creation (chmod-style: 0666)

File descriptors used?	Yes — pipe() returns two FDs	Yes — open() returns an FD, same read()/write()

OS CONCEPT

Both unnamed and named pipes use file descriptors for all operations. The difference is how the FDs are obtained: unnamed pipes use pipe() + fork() inheritance, while named pipes use mkfifo() to create a filesystem entry and then open() to get an FD. Once you have the FD, read() and write() work identically for both.

Creating and Using a Named Pipe

You create a FIFO using the mkfifo() system call, then open it like a regular file using open(). One process opens it for writing (O_WRONLY) and another opens it for reading (O_RDONLY).

NOTE

A named pipe has a critical synchronization property: when you call open() on a FIFO for reading, it BLOCKS until another process opens the same FIFO for writing (and vice versa). This ensures both sides are connected before any data flows. This is automatic — no extra synchronization code needed.

Example 6: FIFO Writer Program

fifo_write.c

```
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>
#include <unistd.h>

int main(void) {
    int fd;
    char buffer[] = "Hello from the writer process!";

    // Create the FIFO (named pipe) with read/write permissions for all
    // 0666 = owner(rw) + group(rw) + others(rw)
    mkfifo("/tmp/myfifo", 0666);

    printf("Writer: Waiting for reader to connect...\n");
    fd = open("/tmp/myfifo", O_WRONLY);   // BLOCKS until reader opens
    printf("Writer: Reader connected! Sending message.\n");

    write(fd, buffer, sizeof(buffer));      // Send the message (same write() as
files!)
```

```
    close(fd);                              // Close the file descriptor

    printf("Writer: Message sent. Exiting.\n");
    return 0;
}
```

Example 7: FIFO Reader Program

fifo_read.c

```
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>
#include <unistd.h>

int main(void) {
    int fd;
    char buffer[512] = {0};

    printf("Reader: Waiting for writer to connect...\n");
    fd = open("/tmp/myfifo", O_RDONLY);    // BLOCKS until writer opens
    printf("Reader: Writer connected! Reading message.\n");

    read(fd, buffer, sizeof(buffer));        // Read the message
    close(fd);                               // Close the file descriptor

    printf("Reader: Received: %s\n", buffer);

    // Clean up the FIFO file from filesystem
    unlink("/tmp/myfifo");
    return 0;
}
```

To test this, you need two terminal windows. Compile and run the writer in one terminal, then the reader in another:

```
# Terminal 1 (Writer)                   # Terminal 2 (Reader)
$ gcc fifo_write.c -o write             $ gcc fifo_read.c -o read
$ ./write                               $ ./read
Writer: Waiting for reader...           Reader: Waiting for writer...
Writer: Reader connected!               Reader: Writer connected!
Writer: Message sent. Exiting.          Reader: Received: Hello from the writer
process!
```

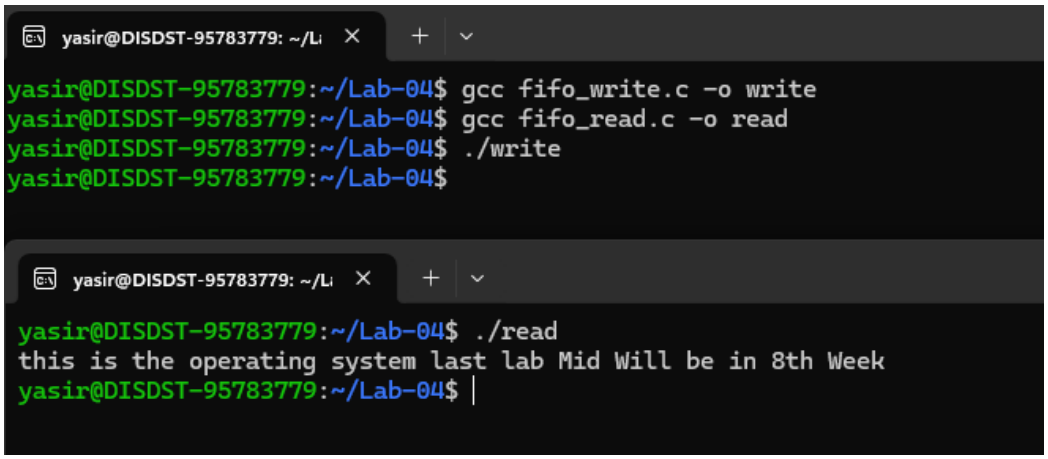

Figure 5: FIFO in action – Writer sends a message (top terminal), reader receives it (bottom terminal).

> **TIP**
> You can also create a named pipe from the command line without writing C code: mkfifo /tmp/myfifo. Then test it with shell commands: echo "hello" > /tmp/myfifo in one terminal and cat /tmp/myfifo in another. This uses the same mechanism — the shell opens the FIFO and uses FDs internally.

Cleaning Up Named Pipes

Unlike unnamed pipes which disappear automatically, named pipes persist on the filesystem. You should always clean them up when done:

```
// In C code:
unlink("/tmp/myfifo");

// From the command line:
$ rm /tmp/myfifo
```

> **WARNING**
> If you run the writer program multiple times without cleaning up, mkfifo() will fail the second time because the file already exists. Either call unlink() in the reader/writer, or add error checking: if mkfifo() returns -1 and errno is EEXIST, the FIFO already exists and you can proceed to open() it.

How It All Connects – The Big Picture

Now you can see how file descriptors are the thread that ties everything together:

Concept	File Descriptor Connection
printf("hello")	Internally calls write(1, "hello", 5) — writes to FD 1 (stdout)
scanf("%d", &x)	Internally calls read(0, ...) — reads from FD 0 (stdin)
./program > file.txt	Shell uses dup2(fd, 1) to redirect FD 1 to the file before exec()
pipe(pipefd)	Creates two FDs: pipefd[0] for reading, pipefd[1] for writing
fork() after pipe()	Child inherits both pipe FDs — now parent and child share the pipe
mkfifo + open()	Creates a named pipe file, open() returns an FD for read/write
cat file | grep word	Shell creates a pipe, redirects cat's stdout (FD 1) to pipe write end, and grep's stdin (FD 0) to pipe read end

OS CONCEPT

The shell pipe operator (|) that you use every day is implemented using exactly the concepts from this lab: the shell calls pipe() to create a pipe, fork()s two children, uses dup2() to redirect FD 1 of the first child to the pipe's write end and FD 0 of the second child to the pipe's read end, then exec()s both commands. Everything you learned in Labs 05 and 06 comes together here!

What's Next: Lab 07

In Lab 07, we will extend IPC further with two more powerful mechanisms:

- Message Queues — structured, prioritized messages between any processes (not just byte streams like pipes).
- Shared Memory — the fastest IPC method, where processes directly access the same memory region.

Both of these still use file descriptors under the hood, so the concepts from this lab are essential preparation.

System Calls – Quick Reference

System Call	Header	Purpose
open(path, flags, mode)	<fcntl.h>	Open a file/device/FIFO, returns new FD
close(fd)	<unistd.h>	Close a file descriptor
read(fd, buf, n)	<unistd.h>	Read up to n bytes from FD into buffer
write(fd, buf, n)	<unistd.h>	Write n bytes from buffer to FD
dup(fd)	<unistd.h>	Duplicate FD (returns new FD pointing to same resource)
dup2(oldfd, newfd)	<unistd.h>	Redirect newfd to point where oldfd points
pipe(int fd[2])	<unistd.h>	Create unnamed pipe: fd[0]=read, fd[1]=write
mkfifo(path, mode)	<sys/stat.h>	Create named pipe (FIFO) at given path
unlink(path)	<unistd.h>	Delete a FIFO or file from the filesystem
fork()	<unistd.h>	Create child process (inherits all FDs)
wait(NULL)	<sys/wait.h>	Wait for child to finish (prevents zombies)
exit(status)	<stdlib.h>	Terminate process (use in child after pipe work)

Lab Tasks

Complete the following tasks. Document your source code, compilation commands, and output screenshots.

Task 1: File Descriptor Exploration

1. Write a C program that opens three different files and prints the file descriptor assigned to each one.
2. Close the second file, then open a new file. What FD number does the new file get? Explain why.
3. Use dup2() to redirect stderr (FD 2) to a file called errors.log. Then call fprintf(stderr, "test error") and verify the message appears in the file, not on the terminal.

Task 2: Unnamed Pipe – Calculator

1. Write a program where the parent sends two numbers and an operator (+, -, *, /) to the child through a pipe.
2. The child reads the data, performs the calculation, and writes the result back to the parent using a SECOND pipe (use two pipes for clean bidirectional communication).
3. The parent prints the final result. Make sure to close all unused pipe ends properly.

Task 3: Shell Pipe Simulation

1. Write a C program that simulates the shell command: ls | wc -l (count the number of files in a directory).
2. Use pipe(), fork(), dup2(), and execl() together. The parent forks two children: Child 1 runs ls with its stdout redirected to the pipe write end. Child 2 runs wc -l with its stdin redirected to the pipe read end.
3. The parent waits for both children to finish. This is how real shells implement the | operator!

Task 4: Named Pipe – Chat Application

1. Create two programs: sender.c and receiver.c that communicate through a named pipe (/tmp/chat_fifo).
2. The sender should continuously read messages from the user (using a while loop with fgets) and send them through the FIFO.
3. The receiver should display each incoming message with a timestamp. Sending the word "quit" should terminate both programs gracefully (use unlink to clean up the FIFO).

Task 5: FD Table Investigation (Bonus)

1. Write a program that opens 5 files, prints all FD numbers, then closes the files with even FD numbers.
2. Open 2 more files and observe which FD numbers the kernel assigns. Are they the ones you freed?
3. Use the /proc/self/fd directory (list it with opendir/readdir or system("ls -l /proc/self/fd")) to see all open FDs of your running process. Take a screenshot of the output and explain each entry.

— End of Lab 06 —